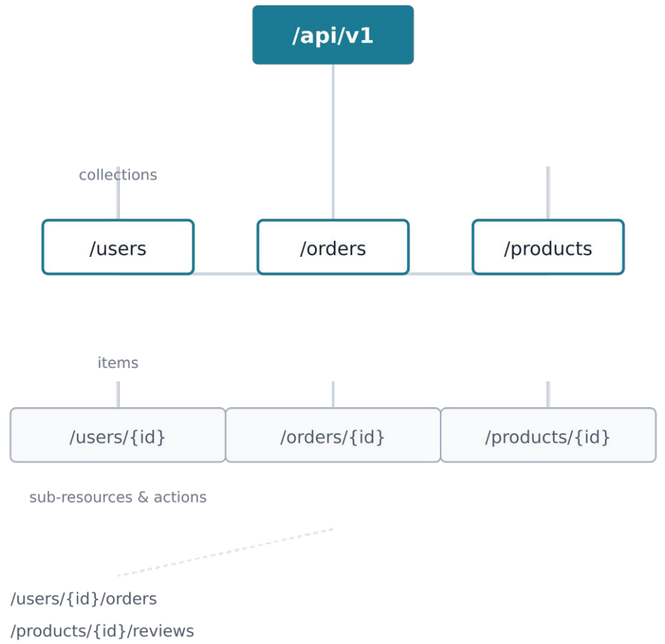
Buổi 3

API Design Principles
và Best Practices

Nguyên tắc thiết kế API hiệu quả · Resource-oriented design · HTTP semantics · Error handling · Pagination & filtering



Mục tiêu

3.1

Giải thích được các nguyên tắc thiết kế API tốt

- Hiểu và trình bày được lý do đằng sau mỗi nguyên tắc
- Áp dụng được cho REST API dạng JSON
- Phương pháp đánh giá: Quizz · Liên quan CLO 2

3.2

Áp dụng naming conventions, đánh giá chất lượng thiết kế API

- Đặt tên endpoint nhất quán với quy ước
- Phát hiện và đề xuất cải thiện các thiết kế có vấn đề
- Phương pháp đánh giá: Quizz · Liên quan CLO 2

Lộ trình buổi học

PHẦN A

Principles & Naming

- Resource-oriented design
- Naming conventions & cấu trúc endpoint
- Ví dụ GitHub API
- **Lab 1:** Thiết kế Blog API

PHẦN B

HTTP & Errors

- HTTP methods & idempotency
- Status codes & semantics
- Error response theo RFC 7807
- **Lab 2:** Flask error handler

PHẦN C

Pagination & Review

- Pagination strategies
- Filtering, sorting, sparse fieldsets
- **Lab 3:** Orders pagination
- Anti-patterns & peer review

Tại sao thiết kế API quan trọng?

Một API được thiết kế tốt là **hợp đồng dài hạn** giữa backend team và người dùng. Khi API được công bố, việc thay đổi nó tốn chi phí gấp nhiều lần so với khi nó chỉ phục vụ một nhóm nhỏ người dùng nội bộ. Theo *Microsoft REST API Guidelines*, các nguyên tắc thiết kế được khuyến nghị giúp giảm **client breakage** và cho phép API mở rộng mà không phá vỡ tích hợp hiện có.

HẬU QUẢ 1

Client breakage khi đổi thiết kế

Twitter công bố API v1.1 sau khi v1 có hàng chục endpoint không nhất quán — hàng nghìn ứng dụng phải sửa lại. Một endpoint viết tắt sai tên khái niệm (như `GET /userList`) khi đổi sang `GET /users` là **breaking change**.

HẬU QUẢ 2

Khó khám phá & học

Một endpoint `/getUserByIdAndStatus` buộc người dùng phải đọc docs mới biết hành vi; cùng lý do đó `/users/{id}?status=...` tự mô tả và có thể *đoán được* đúng hành vi.

Case study

Stripe: consistency compound theo thời gian

Hơn 600 endpoint theo cùng quy ước, ví dụ `POST /v1/customers`, `GET /v1/charges/:id`. Mô hình ổn định giúp tích hợp mới gần như **không tốn chi phí học**.

NGUYÊN TẮC

Consistency > cleverness

Một API nhất quán có thể *dùng được*. Một API không nhất quán buộc mọi tích hợp phải đọc kỹ tài liệu mỗi lần. Nguyên tắc số một của API design là: **mọi quyết định đặt tên phải được lặp lại y hệt ở mọi nơi có thể**.

Resource — danh từ, không phải động từ

URL **định danh tài nguyên** (resources), HTTP method **diễn đạt hành động**. Tài nguyên là *sự vật* trong miền nghiệp vụ — không phải thao tác hay chức năng.

✗ Vi phạm

`GET /getUser?id=123`
Verbs trong URL · query string cho định danh

`POST /createOrder`
Động từ ở path · method POST không khớp tên

`GET /searchProduct?q=phone`
Tìm kiếm nên là filter, không phải resource mới

`POST /users/123/deleteUser`
Động từ lẫn với method, thừa path segment

✓ Tuân thủ

`GET /users/123`
Danh từ ở path · định danh trong path segment

`POST /orders`
Tạo mới tài nguyên *orders*

`GET /products?q=phone`
Tìm là một dạng GET trên collection hiện có

`DELETE /users/123`
Method diễn đạt hành động xóa

Resource là một **danh từ** trong miền nghiệp vụ: User, Order, Product, Invoice, Comment, Review.
Khi một URL cần chứa *tính từ* cho một hành động đặc biệt, hãy cân nhắc nó là một **sub-resource**.
VD: `/users/123/sessions/current` thay vì `/getCurrentSession`.

Quy ước đặt tên endpoint

Một bộ quy ước thống nhất giúp API có thể *đoán trước được*. Sáu quy tắc dưới đây tổng hợp từ *Google API Design Guide*, *Microsoft REST API Guidelines* và thực tiễn của Stripe, GitHub.

1. Lowercase toàn bộ path

URL phân biệt hoa thường nhưng quy ước web là lowercase. Dùng `/users` chứ không phải ~~`/Users`~~.

2. kebab-case cho path, snake_case cho query

`/shopping-cart-items?created_after=2025-01-01`

3. Số nhiều cho collections

`/users`, `/orders`, `/products` — đường số ít hiếm khi hữu ích.

4. Không động từ trong path

Hành động được diễn đạt bằng HTTP method, không phải bằng tên URL.

5. Không đuôi file mở rộng

Dùng `Accept` header để negotiate. Không có ~~`.json`~~ ở URL.

6. Forward slash phân tách quan hệ

`/users/{id}/orders` cho quan hệ "nằm trong".

✗ Không nên

```
GET /UserList/getUserByID?id=10
POST /CreateNewOrder
PUT /updateUser/10?Status=active
GET /api/v1/products.json?sort_by=name
```

Một quy ước nhỏ nhưng lặp lại ở mọi endpoint sẽ tiết kiệm hàng giờ đọc tài liệu cho cả team.

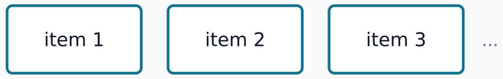
Ba dạng endpoint chính

Mọi endpoint REST nên được phân loại về một trong ba dạng: **collection**, **item**, hoặc **sub-resource**.

Khi nhầm lẫn giữa ba dạng này, URL sẽ phình to và ý nghĩa mơ hồ.

DẠNG 1

Collection




```
GET /users
POST /users
```

Trả về **danh sách** các tài nguyên, hoặc tạo một tài nguyên mới. Thường hỗ trợ filter, sort, paginate.

DẠNG 2

Item (singleton)




```
GET /users/{id}
PATCH /users/{id}
DELETE /users/{id}
```

Tác động lên một **tài nguyên duy nhất** xác định bởi ID. PUT/PATCH dùng để thay thế/cập nhật.

DẠNG 3

Sub-resource




```
GET /users/{id}/orders
POST /users/{id}/orders
```

Quan hệ **cha-con** trong miền. Độ sâu khuyến nghị: tối đa **3 cấp**; sâu hơn ⇒ cần refactor.

Nguyên tắc về độ sâu nesting

2 cấp: lý tưởng · dễ đọc, dễ cache

3 cấp: chấp nhận được · quan hệ phức tạp

≥4 cấp: nên refactor · hoặc dùng filter thay vì nest

✗ /orgs/{o}/teams/{t}/projects/{p}/tasks/{tid}

✓ /tasks?project_id={p} — flatten khi có thể

Case study thực tế

GitHub REST API — phân tích thiết kế

GitHub công bố REST API công khai từ năm 2008 và chọn giữ *ổn định* thay vì thiết kế thường xuyên thay đổi.

Hoạt động được hơn một thập kỷ với **hàng triệu tích hợp**.

QUYẾT ĐỊNH 1

Path mô tả quan hệ sở hữu

```
/repos/{owner}/{repo}/issues
/repos/{owner}/{repo}/issues/{number}
/repos/{owner}/{repo}/issues/{number}/comments
/user/repos
```

`{owner}/{repo}` nằm trong URL thay vì query, vì *định danh repo* cần 2 khóa tự nhiên.

Đúc kết: Khi ID tự nhiên có nhiều phần, đặt chúng trong path.

QUYẾT ĐỊNH 2

Singular resource cho tài nguyên thuộc về user hiện tại

```
/user → thông tin user đang xác thực
/user/repos → repos của user đó
/user/orgs → orgs user đó thuộc về
```

Thay vì `/users/me`, GitHub dùng `/user` (số ít) — vì user ở đây *luôn là một*, đại diện cho ngữ cảnh đang xác thực.

Đúc kết: Singular ổn khi nó là *current actor*.

QUYẾT ĐỊNH 3

Sort + direction bằng query parameters

```
?sort=updated&direction=asc // sắp xếp theo updated, tăng dần
?sort=updated&direction=desc // sắp xếp theo updated, giảm dần
// Với nhiều GitHub REST endpoints, direction mặc định là
desc.
```

QUYẾT ĐỊNH 4

Pagination kết hợp page + per_page, mặc định 30

```
?per_page=50 // max 100
?page=2 // page number bắt đầu từ 1
Link: rel="next", rel="last" // header cho next page URL
```

LAB 1 · 25′ Thực hành · Phần A

Thiết kế resource cho Blog API

BÀI TOÁN

Nền tảng blog đơn giản

Một blog cho phép người dùng đăng **bài viết (posts)**, mỗi bài có **bình luận (comments)** và gắn **thẻ (tags)**. Mỗi user có **hồ sơ** và đăng ký **theo dõi (follow)** tác giả khác. Hãy thiết kế cấu trúc endpoint đầy đủ.

THIẾT KẾ

1. Xác định **resources** trong miền
2. Phân loại collection / item / sub-resource
3. Vẽ sơ đồ cây endpoint và quyết định version segment
4. Triển khai Flask routes cho collection `/posts`

HTTP Methods: semantics và vai trò

RFC 9110 định nghĩa các *safe methods* (không thay đổi state) và *idempotent methods* (gọi nhiều lần cho cùng kết quả). Hiểu rõ hai thuộc tính này giúp thiết kế API không gây hại khi retry tự động xảy ra.

METHOD	SAFE	IDEMPOTENT	VAI TRÒ	REQUEST BODY MẪU
GET	√	√	Đọc tài nguyên. Có thể cache.	— không nên có body
POST	✗	✗	Tạo mới một tài nguyên con.	`{ "title": "...", "body": "..." }`
PUT	✗	√	Thay thế *toàn bộ* tài nguyên.	`{ "title": "...", "body": "...", "tags": [] }`
PATCH	✗	— *	Cập nhật *một phần*.	`{ "title": "Tiêu đề mới" }`
DELETE	✗	√	Xóa tài nguyên.	— không body; idempotent sau lần đầu

* replace một field bằng cùng giá trị có thể idempotent

Tại sao điều này quan trọng: **client và gateway thường tự *retry* khi nhận timeout / network error. Nếu POST `/orders` retry, khách hàng có thể bị tính tiền 2 lần. Đó là lý do PUT và DELETE idempotent, POST không idempotent.**

Safety và Idempotency

SAFE METHOD

Không thay đổi state server

Một method được gọi là *safe* nếu nó được thiết kế để chỉ **truy vấn**, không có hiệu ứng phụ quan sát được ngoài ý muốn client. GET, HEAD, OPTIONS.

IDEMPOTENT METHOD

Gọi N lần = gọi 1 lần

Hiệu ứng cuối cùng trên server là *giống nhau* dù request được gửi 1 lần hay N lần. GET/HEAD/PUT/DELETE idempotent; POST/PATCH thường không.

Kịch bản thực tế · client retry sau timeout

POST /orders · KHÔNG idempotent

Retry tạo ra 2 đơn hàng

```
→ POST /orders 201 Created · order #101
… timeout, không nhận response …
→ POST /orders 201 Created · order #102
```

Hậu quả: khách có thể bị tính tiền 2 lần, 2 bản ghi trong DB.

Khắc phục: dùng *Idempotency-Key* header (xem Stripe).

DELETE /users/42 · idempotent

Gọi 3 lần cho cùng kết quả

```
→ DELETE /users/42 204
→ DELETE /users/42 204 · hoặc 404
→ DELETE /users/42 204 · hoặc 404
```

Hai chiến lược đều đúng:

— trả 204 cho mọi lần (che giấu trạng thái)

— trả 404 sau lần đầu (chính xác hơn)

PUT /users/42 · idempotent

Retry không tạo trạng thái mới

```
→ PUT /users/42 {name: A} → 200
→ PUT /users/42 {name: A} → 200
→ PUT /users/42 {name: A} → 200
```

Người dùng cuối cùng vẫn là **name = A**, không phụ thuộc số lần retry.

STRIPE PATTERN

```
POST /v1/charges Idempotency-Key: a1b2c3d4 // retry cùng key = server trả response đã lưu, không tạo charge mới
```

HTTP Status Codes — 5 lớp ngữ nghĩa

Status code là *phần ngôn ngữ chung* giữa server và client.
Một code chính xác cho phép client phản ứng đúng mà không cần parse body. VD: retry khi 503, refresh token khi 401.

1xx

Informational

Đang xử lý, hiếm khi client gặp trực tiếp.

- **100** Continue
- **101** Switching
- **103** Early Hints

2xx

Success

Request được hiểu và xử lý thành công.

- **200** OK · có body
- **201** Created
- **202** Accepted
- **204** No Content
- **206** Partial (range)

3xx

Redirection

Client cần làm thêm hành động để hoàn tất.

- **301** Moved Permanently
- **302** Found (tạm)
- **304** Not Modified
- **307** Temporary, giữ method

4xx

Client Error

Lỗi do request của client. *Không nên retry* mà không sửa lại request.

- **400** Bad Request
- **401** Unauthorized
- **403** Forbidden
- **404** Not Found
- **409** Conflict
- **422** Unprocessable

5xx

Server Error

Server thất bại xử lý một request hợp lệ. *Có thể retry*.

- **500** Internal Error
- **502** Bad Gateway
- **503** Service Unavailable
- **504** Gateway Timeout

Chi tiết đầy đủ tại RFC 9110 §15 — trang trước liệt kê các code thường gặp nhất trong production.

13 status code thường gặp nhất

Theo khảo sát nội bộ của nhiều công ty (Stripe, Twilio và Cloudflare), gần như toàn bộ traffic của API thành công/lỗi được bao phủ bởi ~13 code dưới đây.

Thành công & Redirect

200 **OK**
Trả về thành công có body. Mặc định cho GET có dữ liệu.

201 **Created**
POST tạo resource mới. **Nên** trả `Location: /users/123` header.

204 **No Content**
Thành công, không có body. Phù hợp cho DELETE, PATCH không trả về entity.

400 **Bad Request**
Cú pháp sai (JSON hỏng, missing required). **có thể dùng 422 để tách semantic validation khỏi lỗi cú pháp**.

401 **Unauthorized**
Chưa xác thực hoặc token hết hạn. `WWW-Authenticate` header bắt buộc.

403 **Forbidden**
Đã xác thực nhưng *không có quyền*. Không phải 401, không phải 404 — nên ẩn sự tồn tại.

Lỗi Client & Server

404 **Not Found**
Resource không tồn tại. Không phân biệt "bị xóa" hay "chưa từng có".

409 **Conflict**
Trạng thái xung đột: trùng email, version cũ, hoặc ETag không khớp.

422 **Unprocessable Entity**
JSON hợp lệ nhưng không qua được quy tắc nghiệp vụ (vd: `age: -5`).

429 **Too Many Requests**
Rate limit vượt. Kèm `Retry-After` header.

500 **Internal Server Error**
Lỗi chung của server. **Không nên lộ stack trace** cho client.

503 **Service Unavailable**
Đang bảo trì hoặc quá tải. Client nên retry với backoff.

Thiết kế error response — RFC 7807 / 9457

Thay vì mỗi service tự định nghĩa cấu trúc lỗi, **RFC 7807** (cập nhật 2023 thành **RFC 9457**) chuẩn hóa định dạng *Problem Details for HTTP APIs*. **Content-Type** nên là `application/problem+json`.

5 trường chuẩn

type	URI tham chiếu mô tả lỗi
title	Tóm tắt ngắn, người đọc hiểu được
detail	Giải thích cụ thể cho lần lỗi này
instance	URI cụ thể nơi xảy ra lỗi
status	HTTP code (số) — trùng với HTTP status code thực sự trong response

Ví dụ · response trả về 403

```
HTTP/1.1 403 Forbidden                    Content-Type: application/problem+json
{
  "type": "https://api.shop.example/probs/out-of-credit",
  "title": "You do not have enough credit.",
  "detail": "Your current balance is 30, but that costs 50.",
  "instance": "/account/12345/msgs/abc",
  "status": 403,
  "balance": 30,
  "accounts": [
    "/account/12345",
    "/account/67890"
  ]
```

LAB 2 · 25′ Thực hành · Phần B

Error handler trả về problem+json

Bài tập: viết một Flask *error handler* thống nhất trả về **problem+json**, kèm exception class

Gợi ý:

- định nghĩa ProblemError(Exception),
- decorator @app.errorhandler(ProblemError), và
- một handler fallback cho HTTPException

Kiểm thử:

- request tới /resources/{id} phải trả 404, status code HTTP là 404,
- body có type/title/detail/status/instance, không lộ stack trace.
- Nếu thiếu Accept hoặc client Accept JSON, vẫn trả problem+json cho lỗi API.
- Nếu gặp exception chưa bắt, trả 500 với message trung tính và log chi tiết server-side.

Ba chiến lược phân trang

Khi một collection có thể chứa hàng triệu bản ghi, *trả hết* trong một response là không khả thi.

Ba chiến lược phổ biến: mỗi chiến lược phù hợp với một kiểu dữ liệu và use case.

OFFSET-BASED

Page + size

```
?page=2&per_page=50
?offset=100&limit=50
```



✓ Ưu điểm

— Dễ hiểu, nhảy trực tiếp đến page N

— Phù hợp cho admin UI, dashboard

✗ Nhược điểm

— Chậm trên data lớn (OFFSET 1M = skip 1M rows)

— Skip-based, không ổn định khi có insert

Dùng bởi: GitHub (page+per_page), Shopify admin

CURSOR-BASED

Cursor-based

```
?limit=50
?cursor="eyJpZCI6MTIzfQ=="
resp.next: "?cursor=..."
```



✓ Ưu điểm

— Có thể ổn định hơn offset nếu cursor encode sort key/snapshot đúng cách

— Hiệu năng tốt trên data cực lớn

✗ Nhược điểm

— Không nhảy tới page N; chỉ next/prev

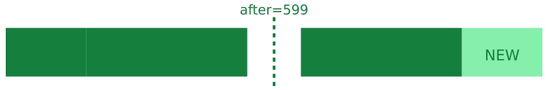
— Cursor phải là opaque, không phân tích được

Dùng bởi: Stripe, Twitter v2, Slack, Discord

KEYSET / SEEK

Khóa sắp xếp + giá trị sau

```
?limit=50
?after_id=1000&sort=id
// hoặc
?after_time=2025-06-01T00:00Z
```



✓ Ưu điểm

— Index scan, tốc độ O(log n) bất chấp offset

— Cursor có thể đọc/hiểu được (debug dễ)

✗ Nhược điểm

— Yêu cầu sort key ổn định (id, created_at)

— Sort phức tạp cần composite key

Dùng bởi: GitHub Events API, Notion, Linear

Filtering, Sorting và Field Selection

Ba khả năng mà collection endpoint nào của API nghiêm túc đều nên hỗ trợ. Thiết kế query string *có chủ đích* không để client phải tự filter ở phía sau.

FILTERING

Thu hẹp kết quả

```
?status=active
?role=admin&status=active
?created_after=2026-01-01
?price_min=100&price_max=500
```

SORTING

Sắp xếp kết quả

```
?sort=created_at // desc
?sort=priority,name // multi-field
```

SPARSE FIELDSETS

Chỉ chọn field cần thiết

```
?fields=id,name,email
?fields=-password_hash // loại trừ
// giảm bandwidth trên mobile
```

Ví dụ

```
GET /products?category=electronics&price_min=100&price_max=500&sort=created_at&fields=id,name,price&page=2
```

Một số quy ước nên nắm được

1. `per_page` có giới hạn trên (vd: 100).
2. Field không tồn tại → **400**.
3. Filter phức tạp: `filter[field]=v`.

LAB 3 · 20′ Thực hành · Phần C

Triển khai `/orders` có cursor pagination

> `GET /orders` với
>
> (1) **cursor**;
>
> (2) **filter:** `status`, `customer_id`;
>
> (3) **sort**;
>
> (4) **sparse fieldsets**.

Test:

- `curl 'localhost:5000/orders?status=paid'`
- `curl 'localhost:5000/orders?limit=5'`
- `curl 'localhost:5000/orders?fields=id,total'`

Hãy thử cursor hỏng để thấy 400.

Cảnh báo

8 anti-patterns phổ biến nhất

- Mỗi lỗi dưới đây đều từng xuất hiện trong các API thực tế của doanh nghiệp lớn.
- Tốt nhất nên được phát hiện và sửa trong quá trình *code review*, trước khi khách hàng phải gánh hậu quả.

#1 · Verbs trong URL

```
✗ POST /createUser / GET /getUsers
√ POST /users / GET /users
```

Method đã diễn đạt hành động — không nhắc lại trong path.

#2 · Trả 200 + error trong body

```
✗ 200 OK + {"success": false, "error": "..."}
√ 4xx/5xx + body có cấu trúc lỗi
```

Phá vỡ retry logic của gateway, monitoring, debugger.

#3 · Case không nhất quán

```
✗ /Users vs /user-profile vs /User_Orders
√ /users /user-profile /user-orders (lowercase + kebab-case)
```

Một API công bố đã có /Users cũ là một breaking change để sửa.

#4 · Số ít / số nhiều lẫn lộn

```
✗ /user (item) vs /orders (collection)
√ /users (tất cả) /users/{id} (một) /me (current actor)
```

Số nhiều cho collection; trừ khi đó là "current actor".

#5 · Auth trong query string

```
✗ GET /data?api_key=abc123&token=...
√ Authorization: Bearer eyJhbGc... // header
```

Token trong URL bị log khắp nơi: access log, Referer, history.

#6 · Đuôi file .json trong URL

```
✗ /users/42.json?include=profile
√ /users/42?include=profile + Accept: application/json
```

Loại bỏ đuôi mở rộng để hỗ trợ nhiều định dạng một cách nhất quán.

#7 · Để lộ DB internal_id ở URL

```
✗ /users?db_id=87&where=1
√ /users/87 hoặc /users/u_abc123 (UUID)
```

Auto-increment IDs cho phép đoán tài nguyên khác; UUID/ULID an toàn hơn.

#8 · Bỏ qua Content negotiation

```
✗ Server luôn trả JSON, không quan tâm Accept
√ If Accept: text/csv → 406 hoặc Content-Type: text/csv
```

Content negotiation giúp API phục vụ đa dạng client thật sự.

CASE STUDY · 15 PHÚT THẢO LUẬN

Refactor API giỏ hàng — bản cũ

Đọc các endpoint và response lỗi

ENDPOINTS · /shop_cart.php

```
POST /addItemToCart?userId=42&productId=99&qty=2

GET /getCartItems

POST /removeItemFromCart

GET /checkOut?userId=42

POST /makeOrder?userId=42

GET /getOrder?orderId=1001

GET /listUserOrders?userId=42

GET /productsList.json?cat=phones

POST /login?username=u&password=p

POST /signup
```

RESPONSE LỖI · getCartItems

```
{ "status": "FAIL",
  "error_code": 5,
  "msg": "out of stock" }
```

HTTP status: 200 ← sai!

Gợi ý: so sánh URL, method, xác thực, tên field, status code và versioning.

Công cụ đánh giá

Checklist peer-review một API

9 tiêu chí để đánh giá nhanh một API trong buổi review chéo. Mỗi mục có 2 lựa chọn *đạt / không đạt*; mỗi mục *không đạt* nên được nêu rõ trong phản hồi.

01
Tài nguyên là danh từ

URLs chỉ chứa danh từ, không động từ. Method diễn đạt action.

02
Naming nhất quán

Lowercase, kebab-case path; snake_case query; plurals cho collection.

03
Status code đúng nghĩa

Mỗi response dùng code phù hợp. Không trả 200 + error trong body.

04
Idempotency rõ ràng

POST cần Idempotency-Key. PUT/DELETE idempotent. Tài liệu hoá rõ.

05
Error response có cấu trúc

RFC 7807 problem+json nhất quán. Có type, title, detail, instance.

06
Pagination rõ ràng

Collection nào cũng có phân trang (cursor/offset). Có giới hạn trên.

07
Filter/Sort đa dạng

Hỗ trợ lọc theo field chính, sort đa field, sparse fieldsets.

08
Authentication & security

Token ở header. Không lộ qua URL. Rate limit rõ ràng.

09
Versioning + deprecation

Có version prefix từ đầu. Có lộ trình ngừng hỗ trợ rõ ràng.

Cách dùng trong lớp

Mỗi nhóm chọn một API công khai (GitHub, Stripe, Spotify, ...) và review bằng 9 tiêu chí. 15 phút chuẩn bị, 5 phút trình bày. Tìm **ít nhất** 2 vấn đề, đề xuất cách sửa.

Tổng kết

1. Resources là **danh từ**, method diễn đạt hành động.
2. Naming lowercase, **kebab-case path**, snake_case query.
3. Endpoint depth ≤ 3; flatten với filter khi sâu hơn.
4. Treat POST as **không-idempotent**; PUT, DELETE idempotent.
5. Status code đúng nghĩa · **không bao giờ** trả 200 + error.
6. Error response theo **RFC 7807** problem+json nhất quán.
7. Pagination: chọn offset, cursor hoặc keyset theo use case.
8. Hỗ trợ filter, sort, sparse fieldsets cho mọi collection.
9. Versioning **từ ngày đầu**; tài liệu deprecation rõ ràng.

BUỔI TIẾP THEO

#4 · API Specification & OpenAPI

Học cách viết OpenAPI spec, tạo Swagger UI, và generate client/server code.

ĐỌC TRƯỚC BUỔI 4

JJ Geewax, *API Design Patterns*, chương 2.

BÀI TẬP VỀ NHÀ

1. Hoàn thiện `/orders` từ Lab 3, viết test cho cursor + sort.
2. Review 1 API công khai bằng 9 tiêu chí, nộp báo cáo.